LAPORAN PRAKTIKUM SISTEM TERBAGI

Instalasi Linux di VirtualBox, Docker & Deployment Aplikasi

Disusun oleh :

Afriza Fastaqimummalka 2341053

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TINGGI TEKNOLOGI MANDALA BANDUNG

2026

BAB 1 — INSTALASI FEDORA LINUX DI VIRTUALBOX

Pada praktikum ini digunakan Fedora Linux 43 KDE sebagai sistem operasi guest yang dijalankan di dalam VirtualBox. VirtualBox dikonfigurasi dengan dua network adapter: Adapter 1 (NAT) untuk koneksi internet, dan Adapter 2 (Host-only) untuk memungkinkan komunikasi antara Windows host dan VM Fedora. Fedora dipilih karena menggunakan DNF sebagai package manager yang modern dan memiliki dukungan Docker yang baik.

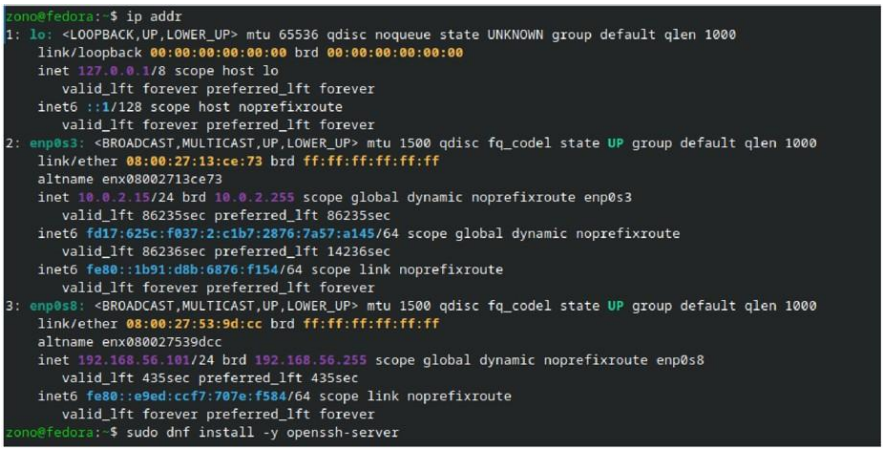
BAB 2 — KONFIGURASI SSH & KONEKSI PUTTY

SSH (Secure Shell) adalah protokol jaringan yang memungkinkan akses dan pengelolaan sistem Linux dari jarak jauh secara aman. PuTTY adalah aplikasi SSH client untuk Windows yang digunakan untuk mengakses Fedora VM dari Windows host. Dengan konfigurasi ini, seluruh perintah Docker dan pengelolaan server dapat dilakukan dari Windows tanpa harus berinteraksi langsung dengan tampilan VM.

a. Cek IP Address Fedora

Langkah pertama adalah mengetahui IP address dari interface Host-only (enp0s8) di Fedora. IP 192.168.56.101 yang ditemukan akan digunakan PuTTY untuk terhubung ke VM, serta digunakan browser Windows untuk mengakses web server.

ip addr



```
zono@fedora:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:13:ce:73 brd ff:ff:ff:ff:ff:ff
    altname enx08002713ce73
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86235sec preferred_lft 86235sec
    inet6 fd17:625c:f037:2:c1b7:2876:7a57:a145/64 scope global dynamic noprefixroute
       valid_lft 86236sec preferred_lft 14236sec
    inet6 fe80::1b91:d8b:6876:f154/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:53:9d:cc brd ff:ff:ff:ff:ff:ff
    altname enx080027539dcc
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
       valid_lft 435sec preferred_lft 435sec
    inet6 fe80::e9ed:ccf7:707e:f584/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
zono@fedora:~$ sudo dnf install -y openssh-server
```

Gambar 2.1 — Hasil perintah ip addr menampilkan IP Host-only 192.168.56.101 pada interface enp0s8

b. Install & Aktifkan OpenSSH Server

OpenSSH Server diinstal di Fedora agar VM dapat menerima koneksi SSH dari luar. Service SSH kemudian diaktifkan dan diset agar otomatis berjalan saat sistem dinyalakan.

sudo dnf install -y openssh-server
sudo systemctl enable sshd
sudo systemctl start sshd
sudo systemctl status sshd

```
Nothing to do.
zono@fedora:~$ sudo systemctl enable sshd
sudo systemctl start sshd
Created symlink '/etc/systemd/system/multi-user.target.wants/sshd.service' → '/usr/lib/systemd/system/sshd.service'.
zono@fedora:~$ sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: disabled)
    Drop-In: /usr/lib/systemd/system/service.d
             └─10-timeout-abort.conf
     Active: active (running) since Sun 2026-04-05 19:40:34 WIB; 6s ago
 Invocation: b2ecd765498941a6be741b70f4d7c828
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 4939 (sshd)
      Tasks: 1 (limit: 5218)
     Memory: 1.2M (peak: 1.5M)
        CPU: 13ms
     CGroup: /system.slice/sshd.service
             └─4939 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 05 19:40:34 fedora systemd[1]: Starting sshd.service - OpenSSH server daemon...
Apr 05 19:40:34 fedora sshd[4939]: Server listening on 0.0.0.0 port 22.
Apr 05 19:40:34 fedora sshd[4939]: Server listening on :: port 22.
Apr 05 19:40:34 fedora systemd[1]: Started sshd.service - OpenSSH server daemon.
```

Gambar 2.2 — SSH server aktif dengan status active (running)

c. Konfigurasi Firewall untuk SSH

Firewall Fedora dikonfigurasi untuk mengizinkan koneksi SSH masuk secara permanen.

sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --reload

```
zono@fedora:~$ sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --reload
Warning: ALREADY_ENABLED: ssh
success
success
```

Gambar 2.3 — Konfigurasi firewall mengizinkan service SSH

d. Koneksi dari Windows via PuTTY

PuTTY diinstall di Windows lalu dikonfigurasi untuk terhubung ke Fedora VM menggunakan IP Host-only. Setelah koneksi berhasil, semua perintah Linux dan Docker diketik melalui jendela terminal PuTTY di Windows tanpa perlu membuka tampilan VM secara langsung.

Host Name : 192.168.56.101
Port : 22
Connection type: SSH

```
>>> Running %posttrans scriptlet: kernel-core-0:6.19.10-200.fc43.x86_64
>>> Selesai %posttrans skrip kecil: kernel-core-0:6.19.10-200.fc43.x86_64
>>> Output skrip:
>>> Generating grub configuration file ...
>>> Adding boot menu entry for UEFI Firmware Settings ...
>>> done
```

Gambar 2.4 — Koneksi PuTTY dari Windows ke Fedora via SSH berhasil

BAB 3 — INSTALASI DOCKER PADA FEDORA LINUX

Docker adalah platform containerisasi yang memungkinkan aplikasi dan seluruh dependensinya dikemas dalam container yang ringan, portabel, dan terisolasi. Pada Fedora, Docker diinstal menggunakan DNF package manager dan dikelola melalui systemd.

a. Install Docker

Instal Docker Engine dari repository resmi Fedora menggunakan DNF package manager.

sudo dnf install -y docker

Gambar 3.1 — Proses instalasi Docker via DNF berhasil

b. Aktifkan & Jalankan Docker Service

Setelah instalasi, Docker diaktifkan sebagai service systemd agar berjalan otomatis saat sistem dinyalakan, kemudian service-nya dijalankan dan statusnya diverifikasi.

sudo systemctl enable docker
sudo systemctl start docker
sudo systemctl status docker

Gambar 3.2 — Status Docker service active (running)

c. Konfigurasi User & Test Docker

User ditambahkan ke grup docker agar dapat menjalankan perintah Docker tanpa sudo. Selanjutnya dilakukan test dengan menjalankan container hello-world untuk memverifikasi instalasi berhasil.

sudo usermod -aG docker $USER
newgrp docker
docker run hello-world

```
[zono@fedora ~]$ sudo usermod -aG docker $USER
newgrp docker
[zono@fedora ~]$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Gambar 3.3 — Pesan "Hello from Docker!" memverifikasi instalasi berhasil

BAB 4 — DEPLOYMENT NGINX WEB SERVER DI DOCKER

Nginx adalah web server berkinerja tinggi yang digunakan untuk melayani konten web statis maupun dinamis. Dengan Docker, Nginx dapat langsung dijalankan dalam container tanpa instalasi manual ke sistem operasi. Web server ini diakses dari browser Windows melalui jaringan Host-only VirtualBox.

1. Pull Image Nginx dari Docker Hub

Image resmi Nginx diunduh dari Docker Hub. Docker secara otomatis mengunduh semua layer yang diperlukan.

docker pull nginx

```
[zono@fedora ~]$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
5e815e07e569: Download complete
cde7a05ae428: Pull complete
bb3d0aa29654: Pull complete
3189680c601f: Pulling fs layer
587e3d84dbb5: Pull complete
ec781dee3f47: Pull complete
510ddf6557d6: Pull complete
669e0ab8e7fa: Download complete
96a6cfe061e0: Download complete
```

Gambar 4.1 — Proses download image Nginx dari Docker Hub

2. Jalankan Container Nginx

Container Nginx dijalankan dengan port forwarding 8080 (host) ke port 80 (container) agar dapat diakses dari browser Windows.

docker run --name my-nginx -p 8080:80 -d nginx

```
[zono@fedora ~]$ docker run --name my-nginx -p 8080:80 -d nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
cde7a05ae428: Pull complete
bb3d0aa29654: Pull complete
587e3d84dbb5: Pull complete
510ddf6557d6: Pull complete
5e815e07e569: Pull complete
ec781dee3f47: Pull complete
3189680c601f: Pull complete
669e0ab8e7fa: Download complete
96a6cfe061e0: Download complete
```

Gambar 4.2 — Container my-nginx berhasil dibuat dan berjalan

3. Buat Folder & File HTML Kustom

Dibuat folder my-website dan file index.html, kemudian diverifikasi menggunakan docker ps bahwa container berjalan dengan status Up.

```
docker ps
mkdir ~/my-website
echo '<h1>Halo dari Nginx Docker di Fedora!</h1>' > ~/my-website/index.html
```

```
[zono@fedora ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                      NAMES
7fdea8952b95   nginx     "/docker-entrypoint.…"   18 seconds ago  Up 17 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    my-nginx
[zono@fedora ~]$ mkdir ~/my-website
echo '<h1>Halo dari Nginx Docker di Fedora!</h1>' > ~/my-website/index.html
```

Gambar 4.3 — docker ps menampilkan container aktif, folder dan file HTML berhasil dibuat

4. Jalankan Container Nginx dengan File HTML Kustom

Container Nginx baru dijalankan dengan volume mount agar menyajikan file HTML yang telah dibuat. Permission folder juga diperbaiki agar Nginx dapat membaca file.

```
chmod 755 ~/my-website
chmod 644 ~/my-website/index.html
docker rm -f my-nginx-custom
docker run --name my-nginx-custom -p 8080:80 \
  -v ~/my-website:/usr/share/nginx/html:ro -d nginx
```

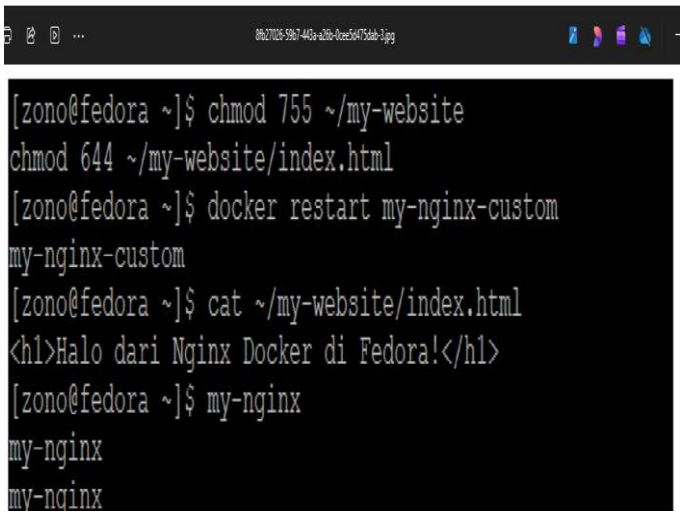
```
[zono@fedora ~]$ docker run --name my-nginx-custom -p 8080:80 -v ~/my-website:/usr/share/nginx/html:ro -d nginx
a5a121f454a237b8e7c0d9c7e660ce37a6da3162d4d75751288ec8652ba0edfa
[zono@fedora ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                      NAMES
a5a121f454a2   nginx     "/docker-entrypoint.…"   7 seconds ago   Up 6 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    my-ngi
```

Gambar 4.4 — Container my-nginx-custom berhasil dibuat ulang dengan volume

5. Restart & Verifikasi Container

Container di-restart untuk memastikan file HTML terbaca dengan benar, lalu diverifikasi statusnya.

```
docker restart my-nginx-custom
docker ps
cat ~/my-website/index.html
```

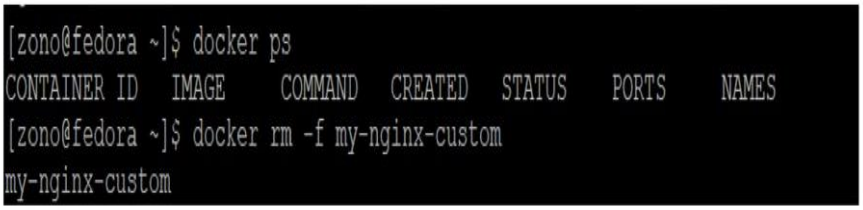


Gambar 4.5 — Container di-restart dan halaman "Halo dari Nginx Docker di Fedora!" berhasil diakses di browser Windows

6. Cek Container & File HTML

Verifikasi container yang berjalan dan memastikan isi file HTML sudah benar menggunakan docker ps dan cat.

docker ps
cat ~/my-website/index.html



Gambar 4.6 — Verifikasi docker ps dan penghapusan container nginx-custom

7. Hasil Nginx di Browser Windows

Buka browser di Windows dan akses IP Fedora beserta port yang dikonfigurasi. Halaman HTML kustom berhasil ditampilkan oleh Nginx yang berjalan di dalam Docker container.

http://192.168.56.101:8080

Gambar 4.7 — Halaman "Halo dari Nginx Docker di Fedora!" berhasil diakses dari browser Windows

8. Stop Container Nginx

Setelah selesai digunakan, container Nginx dihentikan untuk membebaskan port 8080 yang akan digunakan container berikutnya.

docker stop my-nginx-custom

```
[zono@fedora ~]$ docker stop my-nginx-custom
my-nginx-custom
```

Gambar 4.8 — Container my-nginx-custom berhasil dihentikan

BAB 5 — DEPLOYMENT APLIKASI PHP DI DOCKER

Selain Nginx, Docker juga dapat digunakan untuk menjalankan aplikasi PHP tanpa memerlukan XAMPP atau instalasi PHP langsung ke sistem operasi. Image php:apache menggabungkan PHP interpreter dengan Apache web server dalam satu container. File PHP ditulis di Windows, ditransfer ke Fedora menggunakan WinSCP, lalu dijalankan melalui Docker container.

a. Download Image & Jalankan Container PHP+Apache

Image php:apache diunduh dari Docker Hub dan container langsung dijalankan dengan mount folder yang berisi file PHP dari project OOP.

```
docker stop my-nginx-custom
docker run --name my-php -p 8080:80 \
  -v ~/my-website:/var/www/html -d php:apache
```

```
[zono@fedora ~]$ docker run --name my-php -p 8080:80 -v ~/my-website:/var/www/html -d php:apache
Unable to find image 'php:apache' locally
apache: Pulling from library/php
b70e08d69800: Pull complete
81fb1bfad0b3: Pull complete
42f8155a24a4: Pull complete
9c6db6cf7eb2: Pull complete
5551960b4d45: Pull complete
```

Gambar 5.1 — Proses download image php:apache dan container berhasil dijalankan

b. Verifikasi Container PHP Berjalan

Container my-php diverifikasi berjalan dengan status Up menggunakan docker ps, dan memastikan tidak ada konflik port.

```
docker ps
```

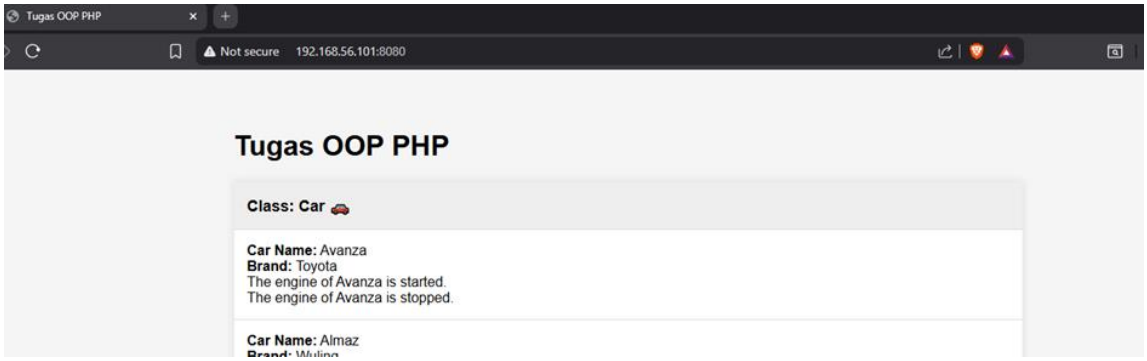
```
[zono@fedora ~]$ nano ~/my-website/index.php
[zono@fedora ~]$ docker run --name my-php -p 8080:80 -v ~/my-website:/var/www/html -d php:apache
```

Gambar 5.2 — Container my-php aktif di port 8080 tanpa XAMPP

c. Akses Aplikasi PHP dari Browser Windows

Buka browser di Windows dan akses IP Fedora di port 8080. Aplikasi PHP OOP berhasil berjalan dan ditampilkan di browser, membuktikan Docker dapat menggantikan fungsi XAMPP.

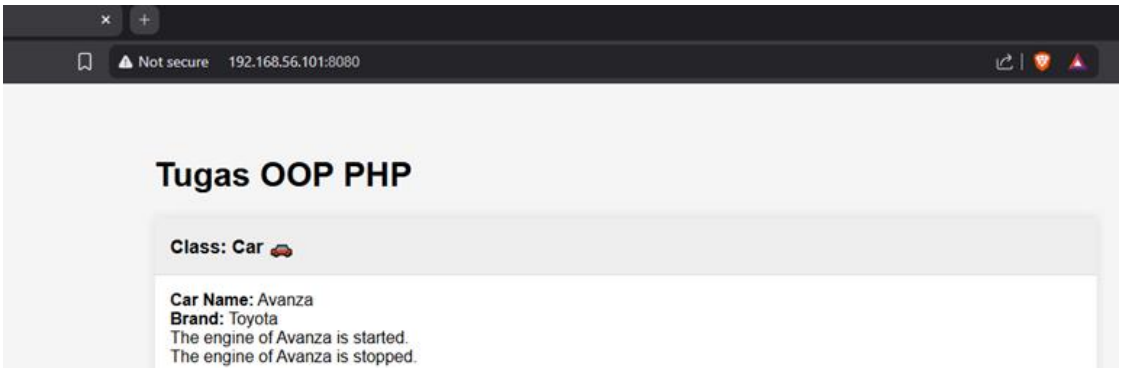
http://192.168.56.101:8080



Gambar 5.3 — Halaman index.php berhasil ditampilkan di browser Windows

d. Hasil Tampilan Aplikasi OOP PHP

Tampilan lengkap aplikasi OOP PHP yang dijalankan melalui container Docker tanpa menggunakan XAMPP. Semua class dan method berjalan dengan baik di dalam container PHP+Apache.



Gambar 5.4 — Aplikasi OOP PHP berhasil diakses dari browser Windows via Docker

BAB 6 — KESIMPULAN

Praktikum ini berhasil mendemonstrasikan implementasi sistem terbagi menggunakan kombinasi VirtualBox, Fedora Linux, Docker, SSH, PuTTY, dan WinSCP. Seluruh rangkaian sistem bekerja sebagaimana mestinya.

1. VirtualBox menyediakan lingkungan virtual untuk menjalankan Fedora Linux 43 KDE di atas Windows.
2. Fedora Linux berfungsi sebagai server tempat Docker Engine berjalan.
3. OpenSSH memungkinkan remote access aman dari Windows ke Fedora.
4. PuTTY digunakan sebagai SSH client di Windows untuk mengontrol Fedora dari jarak jauh.
5. WinSCP memudahkan transfer file PHP dari Windows ke Fedora via SFTP.
6. Docker menjalankan container Nginx dan PHP+Apache secara terisolasi tanpa konflik.
7. Browser Windows berhasil mengakses web server Nginx dan aplikasi PHP OOP yang berjalan di dalam Docker container di Fedora.

Konfigurasi ini mencerminkan workflow nyata seorang DevOps/System Administrator: bekerja dari Windows, mengelola server Linux dari jarak jauh via SSH, dan men-deploy aplikasi menggunakan container Docker. Docker terbukti dapat menggantikan XAMPP dengan cara yang lebih modern, terisolasi, dan mudah dikelola.